

ActPlane: OS-Enforced Agent Harnesses

Paper #XXX
Anonymous Submission

Abstract

Modern coding agents are governed largely by natural-language instructions such as `CLAUDE.md` and `AGENTS.md`. These instructions express real workflow and data-handling constraints, but they are probabilistic: an otherwise cooperative agent may forget them, invoke a shell, spawn a subprocess, or use an SDK path that bypasses a tool-level guard. Existing agent guardrails therefore sit at the wrong layer for constraints that must hold across the agent’s whole execution.

ActPlane is an OS-level harness for AI agents. It compiles declarative information-flow policies into an eBPF/BPF-LSM enforcement backend that tracks typed labels across process, file, and endpoint objects; checks source-to-sink rules at kernel hooks; and returns corrective feedback to the agent so that a failed action can be repaired rather than blindly retried. The system targets cooperative-but-forgetful agents, not only adversarial code. Its semantics therefore distinguish LSM-only pre-operation *block*, tracepoint-compatible *audit*, and harness-level *kill*. This paper describes ActPlane’s policy language, kernel design, feedback bridge, and an evaluation plan that will measure bypass coverage, agent recovery, false positives, overhead, and real-policy coverage. Evaluation result cells are intentionally left as TBDs until the experiments are run.

CCS Concepts: • **Computer systems organization** → *Embedded and cyber-physical systems*; • **Security and privacy** → *Software and application security*; • **Software and its engineering** → *Software safety*.

Keywords: AI agents, eBPF, BPF-LSM, information-flow control, provenance, taint tracking, corrective feedback

ACM Reference Format:

Paper #XXX. 2026. ActPlane: OS-Enforced Agent Harnesses. In *Proceedings of 2026 ACM SIGOPS Annual Technical Conference (ATC '26)*. ACM, New York, NY, USA, 7 pages.

1 Introduction

AI coding agents increasingly operate as long-running processes with access to shells, source trees, package managers, browsers, issue trackers, and external APIs. Project maintainers attempt to steer these agents with instruction files such as `CLAUDE.md` and `AGENTS.md`: do not push directly to main; run tests before committing; never expose secrets; use a specific review tool before destructive operations. These are not simply style preferences. Many are behavioral invariants over the agent’s execution.

The problem is that the dominant enforcement point is still the prompt or the tool-call API. Prompt rules are probabilistic,

and tool-level guards only see actions that pass through the guarded API. A coding agent that forgets a rule can invoke `bash -c`, use `python -c subprocess.run(...)`, run an SDK, or call a syscall through a helper. The agent need not be malicious for the guardrail to fail; it only needs enough autonomy to route around the layer where the rule was encoded.

ActPlane takes the harness boundary below the tool layer. It treats an agent run as an OS execution tree and enforces policies at kernel observation and mediation points. Policies are expressed as typed information-flow rules: sources label processes, files, and endpoints; labels propagate through fork/exec, file reads and writes, and connect-level endpoint edges; sinks declare which labeled subjects may not perform which actions. Enforcement happens through BPF-LSM hooks when the kernel can deny before commit, and through tracepoints when only observation or harness-level termination is available.

This framing differs from both traditional sandboxing and traditional access control. A sandbox usually draws a coarse perimeter around the entire agent. Access control asks whether a principal may touch a resource. An agent harness instead encodes workflow contracts over a cooperative but fallible actor: *if this action violates the contract, fail it and explain how to proceed*. The feedback path matters because a coding agent can often recover: after being told that it must run tests before committing, it can run the tests and retry.

ActPlane’s core technical claim is not that in-kernel provenance enforcement is new. Systems such as CamQuery already showed that labels can be propagated over process, file, and network provenance and used to prevent policy violations [14]. ActPlane’s contribution is to make this mechanism an agent harness: a modern eBPF/BPF-LSM implementation, an agent-oriented rule language, and a corrective feedback loop integrated with contemporary coding agents.

This paper makes four contributions.

- It defines OS-enforced agent harnesses as behavioral invariants plus corrective feedback over an agent’s whole execution, and explains why the enforcement point must be below the tool API for shell/subprocess/SDK bypasses.
- It presents a compact policy language for source/sink/gate rules over process, file, and endpoint objects, including explicit *audit*, *block*, and *kill* effects.
- It describes an implementation that compiles policies into a fixed kernel ABI, propagates labels in eBPF maps, checks rules at tracepoints and BPF-LSM hooks, and

bridges kernel violations into agent feedback files and post-tool hooks.

- It lays out an evaluation plan, derived from real agent instruction files, for measuring bypass coverage, recovery from corrective feedback, false positives, overhead, and expressiveness against tool-layer and kernel baselines.

The paper does not report evaluation numbers until they can be regenerated from the experiment artifacts. The evaluation section specifies the experiments and result tables, with result cells marked TBD.

2 Motivation and Requirements

2.1 Natural-Language Harnesses Are Real

We built a snapshot and analysis pipeline for `CLAUDE.md` and `AGENTS.md` files from popular software projects. The preliminary extraction shows that these files contain many imperative rules, but only a subset are OS-observable behavioral constraints. This distinction is important. A rule such as “never use any” is a coding-style rule and is outside ActPlane’s scope. A rule such as “never push directly to main” or “run tests before committing” has an observable process and syscall footprint and can be enforced as a harness contract.

The paper will use the corpus study for two purposes. First, it grounds the motivation: developers already write behavioral constraints for agents. Second, it derives evaluation workloads: only rules that are observable, expressible, robust, and tempting are converted into policy/task pairs. The final paper should report both the unbiased corpus denominator and the filtered evaluation subset, rather than selecting only examples that ActPlane can handle.

2.2 Why Tool-Layer Guards Are Insufficient

Agent frameworks and permission systems can intercept tool calls and apply deterministic policies [18, 21]. This is useful, but it assumes that effects pass through the guarded tool boundary. A coding agent violates that assumption whenever it invokes a shell, starts a subprocess, loads a library, or talks to an external service from generated code. The same semantic action, for example invoking `git push -force`, can appear as a direct shell command, a nested `bash -c` string, a Python subprocess, or a helper binary. A tool-layer guard sees only the outer tool invocation.

The OS sees the effect itself. Every `exec`, `file open`, `unlink`, `rename`, `truncate`, and `socket connect` crosses a kernel boundary. If a policy is phrased over those effects and over the lineage of the agent process tree, then the policy applies regardless of the high-level route the agent selected.

2.3 Requirements

ActPlane is designed around five requirements.

R1: Whole-execution scope. A rule must apply to the agent’s descendants, not just to the top-level process or the initial tool call.

R2: Cross-object provenance. Many rules are about information flow, not a single event. Reading a secret file should taint the reader; writing a derived file should taint that file; reading the derived file later should taint the next process.

R3: Effect semantics that match the backend. `block` means pre-operation denial and is implemented only by BPF-LSM hooks. Tracepoints do not support `block`; they can report audit rules or terminate kill rules.

R4: Corrective feedback. The harness should not only fail a violating action. It should tell the agent which rule fired, why, and what alternative or precondition can allow progress.

R5: Deployable mechanism. The implementation should use upstream Linux interfaces: eBPF for dynamic loading and BPF-LSM for pre-operation deny, rather than a custom kernel module.

3 Policy Model

ActPlane policies are object-level information-flow rules over three node types: processes, files, and endpoints. A process node represents a Linux task identified by pid and command identity. A file node is currently keyed by path. An endpoint node is currently keyed by IPv4 address. The model is intentionally coarser than byte-level dynamic taint tracking [11]; it trades precision for low-overhead whole-system coverage.

3.1 Labels and Propagation

Each node carries a bitset of labels. A source declaration introduces labels: an `exec` source labels processes that execute a matching binary, a `file` source labels matching files, and an `endpoint` source labels matching network endpoints. Labels propagate through fixed transfer functions:

- `fork`: a child process inherits the parent’s labels;
- `exec`: a process absorbs labels from the executed file and matching `exec` sources;
- `read`: a process absorbs labels from the file it reads;
- `write`: a file absorbs labels from the process writing it;
- `connect`: an endpoint records labels from the connecting process.

This is the provenance invariant ActPlane maintains: if a process carries label `SECRET`, then information from some `SECRET` source has reached that process through observed OS-level flow edges.

3.2 Rules, Gates, and Effects

A rule contains one or more deny clauses. A clause specifies an operation (`exec`, `read`, `write`, `unlink`, `connect`, or `open`), a target pattern, a Boolean expression over labels, and an

```

label AGENT
source SECRET = file "**/.env"

rule no-secret-egress:
  deny connect endpoint "*" if SECRET
  effect block
  reason "secret-derived data must stay local"
  remediation "run the redactor or ask the user"

rule no-agent-git:
  deny exec "**/git" if AGENT
  effect kill
  reason "agent runs must not invoke git"

```

Figure 1. A shortened ActPlane policy. The first rule asks for LSM-only pre-operation deny. The second rule asks for harness-level termination and can fire from tracepoint observation.

optional condition such as a target allow-list, a lineage gate, or an after gate.

Rules have explicit effects.

Audit. The kernel reports the violation and allows the action to proceed. This is useful for soft workflow guidance and for measurement.

Block. A BPF-LSM hook returns `-EPERM` before the operation is committed. block is not supported in tracepoint mode and is not silently downgraded to audit or kill.

Kill. The kernel sends `SIGKILL` to the current task. If a pre-operation LSM hook is active, ActPlane can also return `-EPERM`; from a tracepoint, kill is harness-level termination rather than pre-operation denial.

If several supported rules match the same event, ActPlane chooses the strongest effect in the order `kill > block > audit`. Unsupported effects are ignored by that backend, so a tracepoint `audit` rule can still fire even when an unsupported `block` rule also matches.

3.3 Why This Is a Harness DSL

The language is deliberately not a general-purpose MAC policy. Its primitives map to agent workflows: “after tests,” “through a review tool,” “data derived from untrusted input,” and “read-only sub-agent.” The grammar is small because policy authors should be able to translate natural-language instructions into a few recurring source/sink/gate shapes.

4 Design

Figure 2 summarizes ActPlane’s control flow. The Rust runner discovers a project policy, compiles its embedded DSL into a fixed binary configuration, starts the eBPF loader, seeds the target command’s pid lineage with the AGENT label, and streams violation events back into a feedback file.

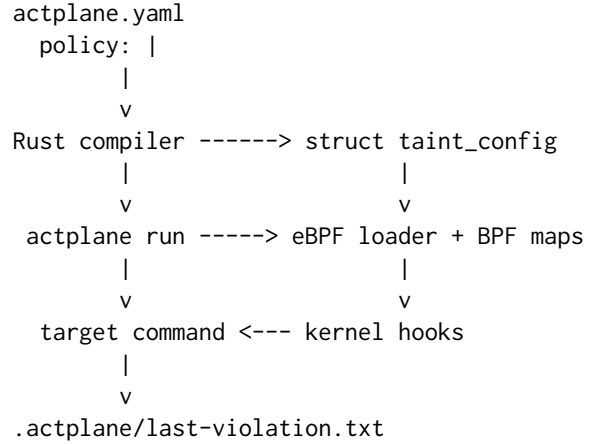


Figure 2. ActPlane pipeline. The collector compiles policy metadata and reasons in userspace, while the kernel enforces and reports rule identifiers.

4.1 Compiler and Kernel ABI

The collector parses the DSL and lowers it into a C-compatible `struct taint_config`. Lowering allocates label bits, expands Boolean expressions into required and forbidden masks, lowers glob patterns into exact, prefix, suffix, or any match kinds, and compiles IPv4 endpoint patterns into network/mask pairs. Human-readable rule names, reasons, and remediation strings remain in Rust metadata; the kernel only receives fixed-size rule tables and emits rule identifiers.

This split keeps the eBPF program verifier-friendly. The kernel program performs bounded loops over fixed-size arrays in read-only data, copies rule entries into stack locals before matching, and uses explicit bounds checks for argv scanning and pattern matching.

4.2 Kernel State

The kernel backend maintains three principal maps:

- `ts_proc`: pid to labels and lineage gates;
- `ts_root/ts_sess`: root process and session-level after gates;
- `ts_file` and `ts_endp`: file and endpoint labels.

Tracepoints maintain lineage and perform observation-compatible checks. BPF-LSM programs perform pre-operation checks for `exec`, file access/mutation, and socket connect. When BPF-LSM is not active in `/sys/kernel/security/lsm`, the loader disables LSM programs and warns that block rules will not fire.

4.3 Unified Event Handling

Each hook normalizes kernel arguments into a small event structure: subject pid, object kind, access kind, backend mode, target display string, optional argv blob, and optional IPv4 address. The shared handler resolves current labels, adds object-source labels for read/connect events, evaluates supported

rules, emits exactly one `TAINT_VIOLATION` event when a supported rule matches, and then applies propagation updates.

This structure is the code-level counterpart to the policy model. File, exec, and network hooks differ only in how they resolve targets and which operations they advertise. Matching and feedback are otherwise centralized.

4.4 Corrective Feedback

The kernel event contains the rule id, effect, target, pid, and flags indicating whether the operation was blocked or killed. The Rust runner maps the rule id back to a rule name, reason, operation, and remediation string, then appends a model-facing payload to `.actplane/last-violation.txt`. The same binary implements `actplane feedback-hook`, a post-tool hook adapter that reads new bytes from that file and returns them as additional context to compatible agents.

The feedback path does not re-evaluate policy in userspace. The kernel is the authority for rule matches. Userspace only formats the reason so a cooperative agent can choose a different route.

5 Implementation Status

ActPlane is implemented as a Rust collector and an eBPF loader/program pair. The current codebase builds a single `actplane` binary that embeds the compiled loader, extracts it at runtime, compiles project policy YAML, and runs a target command under the policy harness.

5.1 Runner

The command `actplane run - <cmd>` discovers `actplane.yaml` upward from the current directory unless `-policy` or `-rule` is specified. The policy YAML contains a policy: | DSL block and an optional feedback path. Run mode requires an `AGENT` label; before the target begins executing, the loader seeds the stopped target pid with that label so all descendants inherit the agent identity.

The runner prepares `.actplane/last-violation.txt` and a hook state file, exports their paths to the child, waits until the loader reports readiness, then continues the target process. This ensures the first target action is observed.

5.2 BPF Backends

The eBPF program attaches to process lifecycle tracepoints for fork, exec, and exit, to syscall tracepoints for observation-compatible file and connect events, and to BPF-LSM hooks for pre-operation enforcement. The loader checks whether the running kernel has the bpf LSM active. If not, it uses tracepoint mode. In tracepoint mode, supported effects are exactly `audit` and `kill`; `block` is unsupported because there is no pre-operation verdict to return.

5.3 Current Limitations

The implementation deliberately exposes several limitations rather than hiding them behind ambiguous semantics.

Path identity. File labels are currently keyed by path hash. This is sufficient for the prototype but should move toward inode/device identity for a stronger claim.

Endpoint identity. Kernel connect matching is numeric IPv4. Hostname, DNS, SNI, and HTTP-layer policies require userspace assist or additional instrumentation.

Exec arguments. `@arg` predicates are currently observed after exec unless an `argv` cache is added before `bprm_check_security`. Thus effect `block` with an `argv` predicate is not available through the tracepoint path; effect `audit` can report and effect `kill` can terminate after exec.

Precision. Labels propagate at object granularity. This can over-taint compared with byte-level DIFT systems. The evaluation must therefore measure false positives and validate declassification/gate paths, not only violation coverage.

5.4 Engineering Invariants

The codebase now follows three invariants that matter for the paper:

- The only kernel output for policy violations is `TAINT_VIOLATION`; ordinary tracing noise is not part of the harness interface.
- Each rule explicitly declares its effect. There is no global fallback that converts `block` into `kill`.
- The post-tool feedback adapter is a transport bridge, not a second policy engine.

6 Evaluation Plan

The evaluation is organized around the following reproducible experiments. Result numbers are not filled in until the corresponding experiment is run with fixed agent versions, logs, and statistical analysis.

6.1 Research Questions

The evaluation targets six questions.

RQ1: Bypass coverage. Does OS-level enforcement catch the same policy violation through direct tool calls, shell strings, subprocesses, SDK paths, and direct syscalls?

RQ2: Corrective feedback. Does semantic feedback improve task completion and reduce repeated violations compared with failure without a reason?

RQ3: False positives. Do benign tasks and sanctioned gate/declassify paths proceed without unnecessary failure?

RQ4: Overhead. What are the per-event and end-to-end costs of label propagation and rule checks?

RQ5: Real-policy coverage. How many real instructions from the corpus are observable, expressible, robust, and temptable?

RQ6: Baselines. Which capabilities are unique relative to tool-layer agent guardrails and kernel provenance/enforcement systems?

6.2 Experiments

6.3 Feedback Conditions

The agent-facing experiment uses four conditions to isolate the contribution of semantic feedback.

- C1: prompt-only. The natural-language rule is present, but there is no ActPlane enforcement.
- C2: audit. ActPlane observes and reports, but does not fail the action.
- C3: fail without ActPlane reason. The violating action fails through `-EPERM` or `SIGKILL`, but the agent receives no structured remediation.
- C4: fail with feedback. The action fails and ActPlane injects the rule reason and remediation through the feedback file and hook bridge.

The main comparison is C3 to C4: the net benefit of the corrective feedback loop. C1 to C2 estimates how often the prompt-only harness fails; C2 to C3 separates observation from action failure.

6.4 Artifact Expectations

The artifact should include policies, trigger scripts, agent prompts, raw logs, statistical notebooks, and a container or VM description pinning kernel, BPF, agent, and model versions. The final paper should avoid reporting any number that cannot be regenerated from these artifacts.

7 Related Work

7.1 In-Kernel Provenance and Information Flow

ActPlane builds on a long line of provenance and information-flow systems. PASS made provenance a storage-layer primitive [13]. Hi-Fi and LPM used LSM hooks to capture trustworthy system provenance [2, 16]. CamFlow captured whole-system provenance streams over processes, files, and sockets [15].

CamQuery is the closest prior system [14]. It executes provenance queries inline in the kernel, propagates labels across process, file, and network edges, and can prevent policy violations. ActPlane does not claim that this mechanism is new. The difference is the domain and substrate: ActPlane targets cooperative AI agents, compiles an agent-oriented source/sink DSL to eBPF/BPF-LSM, and returns corrective feedback to the actor that violated the rule.

TaintDroid, DyTan, libdft, and Panorama established dynamic taint tracking at runtime, binary-instrumentation, or emulation layers [6, 9, 11, 22]. ActPlane uses a much coarser object-level taint. That coarseness is what allows kernel-wide coverage, but it creates an over-taint risk that must be measured.

7.2 eBPF and LSM Enforcement

BPF-LSM, originally proposed through KRSL, lets eBPF programs attach to Linux security hooks and return allow/deny verdicts [19, 20]. ActPlane uses this as its pre-operation back-end. Contemporary eBPF security tools and proposals such as Tetragon, Tracee, and eBPF-PATROL demonstrate practical event capture and enforcement, but generally evaluate per-event predicates or lineage flags rather than typed cross-channel information flow [1, 5, 10].

7.3 Agent Guardrails

AgentSpec and Progent are the closest agent-policy systems at the tool layer [18, 21]. They provide deterministic checks over tool names, arguments, or framework actions. ActPlane complements these systems by moving the enforcement point below the tool API. If all effects pass through the framework, tool-layer policies are sufficient and cheaper. If the agent can execute arbitrary code, spawn subprocesses, or use direct system interfaces, the OS layer is the robust boundary.

FIDES and CaMeL show that typed information-flow and capability separation are useful abstractions for LLM agents [7, 8]. They operate inside the agent loop or scaffolding. ActPlane applies the same style of information-flow reasoning at the OS boundary.

7.4 Agent Instruction Files

Recent empirical work studies `CLAUDE.md`, `AGENTS.md`, and other agent context files: their structure, content, maintenance, and effect on task efficiency [3, 4, 12, 17]. ActPlane's corpus study asks a different question: which instructions are behavioral or flow invariants, and which of those can be observed and enforced below the tool layer?

8 Discussion

8.1 Security Tool or Harness?

ActPlane can express security-relevant policies, such as keeping secret-derived data local. However, its primary target is an agent harness: a deterministic operating contract for a cooperative but forgetful agent. This affects design choices. A classic reference monitor would prioritize pre-operation denial for all security claims. ActPlane also needs audit-only guidance, harness-level termination when pre-operation arguments are unavailable, and remediation text that lets the agent complete the original task through an allowed path.

8.2 Where the Claim Must Stay Honest

The strongest novelty claim is not the taint algorithm. In-kernel label propagation and provenance enforcement are established ideas. ActPlane must be evaluated as a systems contribution that combines those ideas with a new workload and interface: autonomous agents with rich tool access, natural-language policy sources, and corrective feedback.

The final paper should be careful about three boundaries.

Table 1. Planned evaluation matrix. Result fields remain TBD.

Experiment	Method	Primary metrics	Result status
E1: Cross-path coverage	Run the same forbidden action through tool call, nested shell, subprocess, SDK/helper, and direct syscall triggers. Compare against tool-layer baselines.	Coverage matrix over policy by path; feedback delivery.	TBD
E2: Feedback loop	Run Codex and Claude-style agents under prompt-only, audit, fail-without-reason, and fail-with-feedback conditions.	Task completion, repeated violations, repair iterations, token/turn cost.	TBD
E3: False positives	Run allowed tasks and sanctioned paths: redaction, test-before-commit, confirmation, and lineage mediation.	False positive rate, allowed-path success.	TBD
E4: Overhead	Microbench fork/exec/read/write/connect and macrobench real agent tasks with and without ActPlane.	p50/p99 latency, wall time, memory/map usage, rule-table scaling.	TBD
E5: Corpus coverage	Double-code real instruction-file rules with D1–D7, report agreement, and derive policies/tasks mechanically.	Observable, expressible, robust, temptable fractions; Cohen’s kappa.	TBD
E6: Baseline comparison	Compare against CamQuery, Tetragon, Progent, AgentSpec, and built-in agent sandboxes where runnable.	Layer, enforce/detect, cross-channel taint, bypass resistance, feedback.	TBD

Block is LSM-only. Tracepoint observation cannot be described as block. The current implementation correctly ignores block rules in tracepoint mode and warns the operator.

Prompt files are aspirational. A corpus of instruction files shows what developers want agents to do. It does not by itself prove how often agents violate those rules. That is the job of the agent evaluation.

Coarse taint can over-approximate. Object-level labels are practical, but they may carry more provenance than a byte-level system would. The evaluation must measure false positives and demonstrate useful declassification and gate patterns.

9 Conclusion

ActPlane moves agent harness enforcement below the tool layer. It compiles agent-oriented information-flow policies into eBPF/BPF-LSM checks, tracks labels across process, file, and endpoint objects, and returns kernel-detected violations as corrective feedback to the agent. The evaluation plan is designed to test bypass coverage, feedback-driven recovery,

false positives, overhead, and coverage over real agent instruction files without reporting numbers before they can be reproduced.

References

- [1] Aqua Security. 2026. Tracee: Linux Runtime Security and Forensics using eBPF. <https://github.com/aquasecurity/tracee>.
- [2] Adam Bates, Dave Tian, Kevin R. B. Butler, and Thomas Moyer. 2015. Trustworthy Whole-System Provenance for the Linux Kernel. In *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, Washington, DC, 319–334. <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/bates>
- [3] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. 2025. Agent READMEs: An Empirical Study of Context Files for Agentic Coding. arXiv:2511.12884. <https://arxiv.org/abs/2511.12884>
- [4] Worawalan Chatlatanagulchai, Kundjanasith Thonglek, Brittany Reid, Yutaro Kashiwa, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, and Hajimu Iida. 2025. On the Use of Agentic Coding Manifests: An Empirical Study of Claude Code. arXiv:2509.14744. <https://arxiv.org/abs/2509.14744>
- [5] Cilium Project. 2026. Tetragon: eBPF-based Security Observability and Runtime Enforcement. <https://tetragon.io/>.

- [6] James Clause, Wanchun Li, and Alessandro Orso. 2007. Dytan: A Generic Dynamic Taint Analysis Framework. In *Proceedings of the 2007 International Symposium on Software Testing and Analysis*. Association for Computing Machinery, London, United Kingdom, 196–206. doi:10.1145/1273463.1273490
- [7] Manuel Costa, Boris Koepf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Beguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643. <https://arxiv.org/abs/2505.23643>
- [8] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramer. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813. <https://arxiv.org/abs/2503.18813>
- [9] William Enck, Peter Gilbert, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. 2010. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*. USENIX Association, Vancouver, Canada, 393–407. <https://www.usenix.org/conference/osdi10/taintdroid-information-flow-tracking-system-realtime-privacy-monitoring>
- [10] Sangam Ghimire, Nirjal Bhurtel, Roshan Sahani, and Sudan Jha. 2025. eBPF-PATROL: Protective Agent for Threat Recognition and Overreach Limitation using eBPF in Containerized and Virtualized Environments. arXiv:2511.18155. <https://arxiv.org/abs/2511.18155>
- [11] Vasileios P. Kemerlis, Georgios Portokalidis, Kangkook Jee, and Angelos D. Keromytis. 2012. libdft: Practical Dynamic Data Flow Tracking for Commodity Systems. In *Proceedings of the 8th ACM SIGPLAN/SIGOPS Conference on Virtual Execution Environments*. Association for Computing Machinery, London, United Kingdom, 121–132. doi:10.1145/2151024.2151042
- [12] Jai Lal Lulla, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M. Zhang, Sebastian Baltes, and Christoph Treude. 2026. On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents. arXiv:2601.20404. <https://arxiv.org/abs/2601.20404>
- [13] Kiran-Kumar Muniswamy-Reddy, David A. Holland, Uri Braun, and Margo Seltzer. 2006. Provenance-Aware Storage Systems. In *2006 USENIX Annual Technical Conference (USENIX ATC 06)*. USENIX Association, Boston, MA, 43–56. <https://www.usenix.org/conference/2006-usenix-annual-technical-conference/provenance-aware-storage-systems>
- [14] Thomas F. J.-M. Pasquier, Xueyuan Han, Thomas Moyer, Adam Bates, Olivier Hermant, David Eysers, Jean Bacon, and Margo Seltzer. 2018. Runtime Analysis of Whole-System Provenance. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, Toronto, Canada, 1601–1616. doi:10.1145/3243734.3243776
- [15] Thomas F. J.-M. Pasquier, Jatinder Singh, David Eysers, and Jean Bacon. 2017. CamFlow: Managed Data-Sharing for Cloud Services. *IEEE Transactions on Cloud Computing* 5, 3 (2017), 472–484. doi:10.1109/TCC.2015.2489211
- [16] Devin J. Pohly, Stephen McLaughlin, Patrick McDaniel, and Kevin Butler. 2012. Hi-Fi: Collecting High-Fidelity Whole-System Provenance. In *Proceedings of the 28th Annual Computer Security Applications Conference*. Association for Computing Machinery, Orlando, FL, 259–268. doi:10.1145/2420950.2420989
- [17] Helio Victor F. Santos, Vitor Costa, Joao Eduardo Montandon, and Marco Tulio Valente. 2025. Decoding the Configuration of AI Coding Agents: Insights from Claude Code Projects. arXiv:2511.09268. <https://arxiv.org/abs/2511.09268>
- [18] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703. <https://arxiv.org/abs/2504.11703>
- [19] KP Singh. 2019. Kernel Runtime Security Instrumentation. Linux Plumbers Conference. <https://lpc.events/event/4/contributions/454/>
- [20] The Linux Kernel Documentation. 2021. LSM BPF Programs. https://www.kernel.org/doc/html/v5.15/bpf/bpf_lsm.html
- [21] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666. <https://arxiv.org/abs/2503.18666>
- [22] Heng Yin, Dawn Song, Manuel Egele, Christopher Kruegel, and Engin Kirda. 2007. Panorama: Capturing System-wide Information Flow for Malware Detection and Analysis. In *Proceedings of the 14th ACM Conference on Computer and Communications Security*. Association for Computing Machinery, Alexandria, VA, 116–127. doi:10.1145/1315245.1315261